KU LEUVEN

Labo 2 - CUDA

Sander De Stercke (MEIECFS)
Isaak Vervenne (MEISWE)

Februari 2026

1 Wat is er gerealiseerd?

In dit labo worden drie verschillende methoden onderzocht om de maximale waarde in een (1D) array te vinden. Concreter implementeren we "max detection"met behulp van:

- Een sequentiële for-loop op de CPU.
- Een GPU kernel die atomic operations gebruikt.
- Een GPU kernel die reduction gebruikt.

Na de implementaties van de drie methoden, worden de prestaties van deze methoden vergeleken op basis van een snelheidstest om de afhankelijkheid van de executietijd op de lengte van de input array te beoordelen. Dit is een manier om de efficiëntie van de verschillende methoden te bepalen.

2 Problemen

Deze opgave ging vrij vlot, Het grootste aandeel van onze tijd ging naar het programmeren van de verschillende methoden en de uitvoering van de snelheidstests. Waarbij de meeste tijd ging naar het oplossen van arrays groter dan 2048 als ook het implementeren van de atomic max detection.

3 Vragen

3.1 Is het resultaat correct?

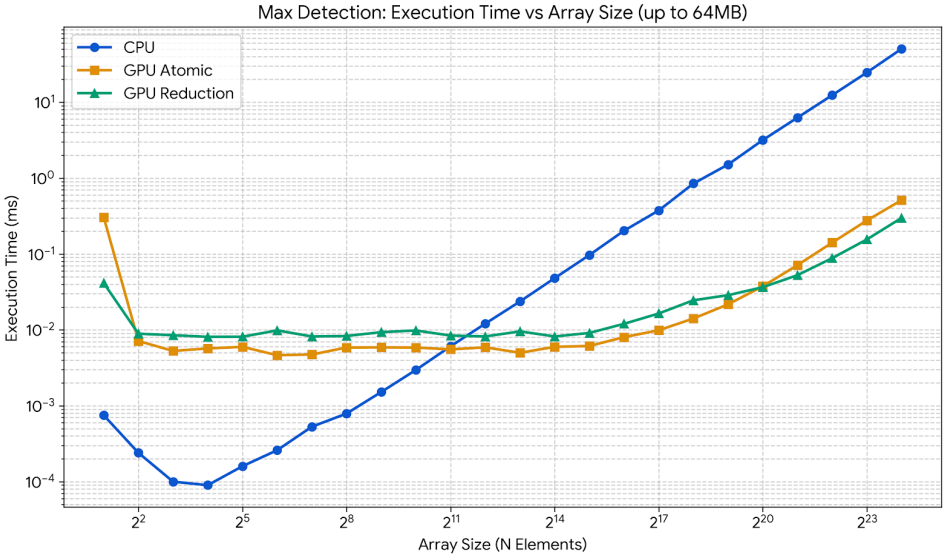
Onze resultaten zijn weldegelijk correct. Telkens werd de hoogst mogelijke waarde in de array gevonden, wat overeenkomt met de baseline die door de CPU werd berekend. We hebben dit ook gecontroleerd door de code aan te passen zodat de maximale waarde niet telkens de limiet van 9999 was.

3.2 Hoe verloopt de executietijd als functie van arraygrootte?

Zoals verwacht neemt de executietijd toe naarmate de arraygrootte toeneemt. Dit is ook wat we kunnen concluderen uit de metingen die we hebben uitgevoerd. De metingen zijn gevisualiseerd in de grafieken hieronder.



Figuur 1: Executietijd vs Arraygrootte (Tot N=1024)



Figuur 2: Executietijd vs Arraygrootte (Opschaling tot 64MB)

De CPU implementatie heeft een lineaire tijdscomplexiteit (O(n)). Deze implementatie is de snelste voor kleinere arraygroottes, maar de executietijd neemt aanzienlijk toe naarmate de arraygrootte toeneemt. Tussen de GPU implementaties is de atomic operation implementatie de snelste voor kleinere arraygroottes, maar voor heel grootte arrays neemt de overhead van synchronisatie bij atomic operations toe, waardoor de reduction implementatie een snellere executietijd heeft.

3.3 Wat gebeurt er als de arraygrootte groter is dan 2048?

Een enkel CUDA thread block is gelimiteerd tot maximaal 1024 threads. Als de input groter is dan 2048 (wat meer dan 1024 elementen per block betekent na de initiële halvering), kan één enkele block met reduction niet de volledige array verwerken.

Om een correcte berekening te verzekeren en extra synchronisatiestappen in te bouwen, moeten we een multi-stage (of multi-block) reduction toepassen. Elk block berekent een lokaal maximum en schrijft dit naar een tijdelijke array. Omdat CUDA geen globale synchronisatie tussen blocks toestaat tijdens de kernel executie, fungeert het beëindigen van de kernel launch als onze synchronisatiestap. Daarna lanceren we een tweede kernel om het uiteindelijke maximum uit die tijdelijke array te reduceren.

3.4 Overweeg het oproepen van een kernel met veel minder threads dan dat er data elementen zijn. Hoe verzeker je dat alle elementen verwerkt worden?

Om te garanderen dat alle elementen verwerkt worden wanneer we bijvoorbeeld maar een tiende van het benodigde aantal threads lanceren, maken we gebruik van een zogenaamde "grid-stride loop".

In plaats van elke thread slechts één element te laten controleren, laten we elke thread over de array itereren met een stapgrootte (stride) gelijk aan het totale aantal threads in de grid (‘blockDim.x * gridDim.x‘). Hierdoor verwerkt elke thread meerdere elementen sequentieel en wordt een lokaal maximum opgebouwd, voordat de eigenlijke block-reduction via shared memory begint.

3.5 Hoe behandel je een 2D array?

Een 2D array kan in het geheugen voorgesteld worden als een platte, aaneengesloten 1D array ("flattened"). Je kan dan simpelweg de lineaire 1D index berekenen met de formule ‘index = rij * breedte + kolom‘. Hierdoor kunnen we exact dezelfde 1D reduction kernels hergebruiken voor 2D data, zonder de logica te hoeven aanpassen.

4 Code

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <limits.h>
#include <algorithm>
#include <chrono>
#include <iomanip>

int cpu_max(int *arr, int n) {
    int max_val = INT_MIN;
    for (int i = 0; i < n; i++) {
        if (arr[i] > max_val) max_val = arr[i];
    }
    return max_val;
}

__global__ void gpu_atomic_max(int *d_arr, int *d_result, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(d_result, d_arr[idx]);
    }
}

__global__ void gpu_reduction_max(int *d_arr, int *d_out, int n) {
    extern __shared__ int sdata[];

    unsigned int tid = threadIdx.x;
    unsigned int global_idx = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int stride = blockDim.x * gridDim.x;

    int my_max = INT_MIN;
    for (int i = global_idx; i < n; i += stride) {
        my_max = max(my_max, d_arr[i]);
    }

    sdata[tid] = my_max;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = max(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }

    if (tid == 0) {
        d_out[blockIdx.x] = sdata[0];
    }
}

int main() {
    // Force CUDA context initialization to avoid first-run penalty
    cudaFree(0);

    // Max size: 16,777,216 elements (64 MB)
    int max_N = 1 << 24;
    size_t max_size = max_N * sizeof(int);

    // 1. Allocate all memory upfront
    int *h_arr = (int*)malloc(max_size);
```

```
60      int *h_result = (int*)malloc(sizeof(int));
61
62      // Fill the massive array once
63      for (int i = 0; i < max_N; i++) h_arr[i] = rand() % 10000;
64
65      int *d_arr, *d_result_atomic, *d_partial_results, *d_final_result;
66      cudaMalloc(&d_arr, max_size);
67      cudaMalloc(&d_result_atomic, sizeof(int));
68
69      int threadsPerBlock = 256;
70      int max_blocks = 1024; // Cap blocks to 1024 for the 2nd reduction pass
71
72      cudaMalloc(&d_partial_results, max_blocks * sizeof(int));
73      cudaMalloc(&d_final_result, sizeof(int));
74
75      // Copy the entire 64MB array to the device once
76      cudaMemcpy(d_arr, h_arr, max_size, cudaMemcpyHostToDevice);
77
78      cudaEvent_t start, stop;
79      cudaEventCreate(&start); cudaEventCreate(&stop);
80
81      // Print Header
82      std::cout << "
            ========================================================================\n";
83      std::cout << std::setw(10) << "N" << "␣|␣"
84                << std::setw(9) << "Size␣(MB)" << "␣|␣"
85                << std::setw(12) << "CPU␣(ms)" << "␣|␣"
86                << std::setw(13) << "Atomic␣(ms)" << "␣|␣"
87                << std::setw(14) << "Reduction␣(ms)" << "\n";
88      std::cout << "
            ========================================================================\n";
89
90      // 2. Loop through sizes, starting from 2 up to 16,777,216
91      for (int N = 2; N <= max_N; N *= 2) {
92          float size_mb = (N * sizeof(int)) / (1024.0 * 1024.0);
93
94          // --- CPU ---
95          auto start_cpu = std::chrono::high_resolution_clock::now();
96          int cpu_res = cpu_max(h_arr, N);
97          auto end_cpu = std::chrono::high_resolution_clock::now();
98          std::chrono::duration<float, std::milli> cpu_ms = end_cpu - start_cpu;
99
100         // --- GPU Atomic ---
101         int initial_min = INT_MIN;
102         cudaMemcpy(d_result_atomic, &initial_min, sizeof(int), cudaMemcpyHostToDevice
                );
103
104         cudaEventRecord(start);
105         int total_atomic_blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
106         gpu_atomic_max<<<total_atomic_blocks, threadsPerBlock>>>(d_arr,
                d_result_atomic, N);
107         cudaEventRecord(stop);
108         cudaEventSynchronize(stop);
109
110         float atomic_ms = 0;
111         cudaEventElapsedTime(&atomic_ms, start, stop);
112
113         // --- GPU Multi-Stage Reduction ---
114         int current_blocks = std::min((N + threadsPerBlock - 1) / threadsPerBlock,
                max_blocks);
115         if (current_blocks == 0) current_blocks = 1;
```

```
116
117         cudaEventRecord(start);
118
119         // PASS 1
120         int sharedMemSize = threadsPerBlock * sizeof(int);
121         gpu_reduction_max<<<current_blocks, threadsPerBlock, sharedMemSize>>>(d_arr,
                d_partial_results, N);
122
123         // PASS 2
124         int threadsPass2 = 1024;
125         int sharedMemSize2 = threadsPass2 * sizeof(int);
126         gpu_reduction_max<<<1, threadsPass2, sharedMemSize2>>>(d_partial_results,
                d_final_result, current_blocks);
127
128         cudaEventRecord(stop);
129         cudaEventSynchronize(stop);
130
131         float reduction_ms = 0;
132         cudaEventElapsedTime(&reduction_ms, start, stop);
133
134         // Print Row
135         std::cout << std::setw(10) << N << "␣|␣"
136                   << std::setw(9) << std::fixed << std::setprecision(4) << size_mb <<
                        "␣|␣"
137                   << std::setw(12) << std::fixed << std::setprecision(5) << cpu_ms.
                       count() << "␣|␣"
138                   << std::setw(13) << std::fixed << std::setprecision(5) << atomic_ms
                        << "␣|␣"
139                   << std::setw(14) << std::fixed << std::setprecision(5) <<
                       reduction_ms << "\n";
140     }
141     std::cout << "
            ========================================================================\n";
142
143     // Cleanup
144     cudaFree(d_arr); cudaFree(d_result_atomic); cudaFree(d_partial_results); cudaFree
            (d_final_result);
145     free(h_arr); free(h_result);
146     cudaEventDestroy(start); cudaEventDestroy(stop);
147
148     return 0;
149 }
```

Listing 1: Multi-Stage Reduction & Atomic Max Detection (Met Grid-Stride Loop)